

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	Kanmani L
Name	192511414

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

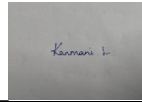
Dave's Taxonomy: Articulation (Level 4)
Question Count: 10

Total Marks: 25

Code of Conduct

I Kanmani L/192511414 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: _____

Assessment Tasks

Mock Interview

Answer the following interview-oriented questions clearly and concisely. Support your responses with architecture-level reasoning wherever appropriate.

1. Explain the difference between HDFS and traditional file systems.
2. Why is replication important in distributed storage systems?
3. How does MapReduce divide work between map and reduce phases?
4. What role does YARN play in large-scale Hadoop processing?
5. Differentiate NAS and SAN in an enterprise data environment.
6. What is the purpose of ETL in a big data pipeline?
7. How does Apache NiFi help in data integration workflows?
8. What are the key failure points in a Hadoop cluster and how would you handle them?
9. How would you design a secure data ingestion pipeline for a large organization?
10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Mock Interview Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Knowledge	25%	Shows deep and accurate technical understanding	Good technical understanding	Basic understanding	Weak understanding
Problem Solving	20%	Responds with strong architecture-level reasoning	Reasonable reasoning	Basic reasoning	Weak reasoning
Communication	20%	Answers are confident, precise, and professional	Mostly clear communication	Moderate clarity	Weak communication
Practical Awareness	20%	Connects concepts to real-world implementation well	Some practical relevance	Limited practical relevance	No practical linkage
Confidence and Professionalism	15%	Highly confident and professional	Generally professional	Moderately professional	Low confidence and professionalism

1. Explain the difference between HDFS and traditional file systems.

Answer:

HDFS (Hadoop Distributed File System) is designed to store and process very large datasets across multiple machines, whereas traditional file systems generally store data on a single server or storage system.

- HDFS: Distributed across multiple DataNodes.
- Traditional file system: Usually depends on a single machine or centralized storage.
- HDFS provides automatic replication and fault tolerance.
- It is optimized for high-throughput, sequential access to large files rather than frequent small-file operations.

Architecture view:

In HDFS, a NameNode manages metadata, while multiple DataNodes store actual data blocks. This distributed architecture allows storage capacity and processing to scale by adding more nodes.

2. Why is replication important in distributed storage systems?

Answer:

Replication means maintaining multiple copies of data blocks on different nodes. It is important because individual machines can fail in a distributed system.

For example, if a data block has a replication factor of 3, three copies are stored on different nodes. If one node fails, another copy can still serve the data.

Benefits:

- Fault tolerance
- High data availability
- Protection against node failures
- Improved read performance
- Automatic recovery through re-replication

Architecture view:

Replication removes a single point of failure at the data-block level and allows the system to continue operating even when individual nodes fail.

3. How does MapReduce divide work between map and reduce phases?

Answer:

MapReduce divides a large data-processing task into two major phases:

Map phase:

- Input data is divided into splits.
- Multiple mapper tasks process these splits in parallel.
- Each mapper produces intermediate key-value pairs.

Shuffle and Sort:

- Hadoop groups intermediate data according to keys.
- The grouped data is sent to the appropriate reducers.

Reduce phase:

- Reducers process values associated with each key.
- They aggregate or transform the data and generate the final output.

Example:

For counting words, mappers produce (word, 1), and reducers add all values for the same word to produce (word, total count).

Architecture view:

The process enables parallel and distributed processing across many machines.

4. What role does YARN play in large-scale Hadoop processing?

Answer:

YARN stands for Yet Another Resource Negotiator. It acts as the resource management and job scheduling layer of Hadoop.

Its main responsibilities are:

- Allocating CPU and memory resources.
- Scheduling applications.
- Monitoring running applications.
- Managing cluster resources among multiple users and jobs.

Architecture view:

YARN mainly consists of:

- ResourceManager: Manages overall cluster resources.
- NodeManager: Manages resources and containers on individual nodes.
- ApplicationMaster: Manages a specific application.

This separation allows Hadoop to support different processing frameworks instead of relying only on MapReduce.

5. Differentiate NAS and SAN in an enterprise data environment.

Feature	NAS	SAN	
Full form	Network Attached Storage	Storage Area Network	
Access	File-level	Block-level	
Protocols	NFS, SMB	Fibre Channel, iSCSI	
Appears as	Shared files/folders	Raw storage devices	
Management	Relatively simple	More complex	
Common use	File sharing, backups	Databases, enterprise applications	

Architecture view:

NAS provides shared files through a network, while SAN provides high-performance block storage that servers can use like locally attached disks.

Interview point:

NAS is generally preferred for file sharing, while SAN is commonly used where high-performance and low-latency block storage is required.

6. What is the purpose of ETL in a big data pipeline?

Answer:

ETL stands for Extract, Transform, Load. It prepares data from different sources before storing it in an analytics or data-storage system.

- Extract: Collect data from databases, applications, files, APIs, sensors, etc.
- Transform: Clean, validate, filter, standardize, and format the data.
- Load: Store the processed data in a data warehouse, data lake, or analytics system.

Architecture view:

Data Sources → Extract → Transform → Load → Analytics

ETL ensures that downstream analytics receives consistent, clean, and usable data.

7. How does Apache NiFi help in data integration workflows?

Answer:

Apache NiFi is a data-flow and integration platform used to move and process data between different systems.

- Collect data from multiple sources.
- Route data based on conditions.

- Transform and filter data.
- Convert between data formats.
- Connect databases, APIs, files, IoT devices, and messaging systems.
- Monitor data flow and provide provenance information.

Architecture view:

Sources → NiFi Processors → Routing/Transformation → Target Systems

For example, NiFi can collect sensor data, validate it, convert it into JSON, filter invalid records, and send valid data to Kafka, HDFS, or a database.

8. What are the key failure points in a Hadoop cluster and how would you handle them?

Answer:

Important failure points include DataNodes, NameNode, network connectivity, disks, and resource-management components.

DataNode failure:

Use HDFS replication so another DataNode can provide the required block. HDFS can automatically re-replicate missing copies.

NameNode failure:

Use NameNode High Availability (HA) with Active and Standby NameNodes.

Disk failure:

Use multiple disks, monitoring, and replication.

Network failure:

Use redundant network paths and monitor network health.

YARN/resource failure:

Use appropriate resource allocation, queue management, and monitoring.

Architecture view:

A reliable Hadoop cluster uses replication + HA + monitoring + automatic recovery rather than depending on a single machine.

9. How would you design a secure data ingestion pipeline for a large organization?

Answer:

I would design the pipeline with security at every stage rather than securing only the final storage system.

A possible architecture is:

Data Sources → Secure Ingestion Layer → Validation → Encryption → Processing → Secure Storage → Analytics

Key measures include:

1. Authentication – Verify the identity of users and data sources.
2. Encryption in transit – Use TLS for data moving between systems.
3. Encryption at rest – Encrypt sensitive stored data.
4. Authorization – Use role-based access control (RBAC).
5. Data validation – Reject malformed or suspicious data.
6. Audit logging – Record who accessed or modified data.
7. Data masking – Protect sensitive fields such as personal information.
8. Monitoring – Detect unusual access or ingestion behavior.

Architecture-level reasoning:

Security should follow a defense-in-depth approach, so failure of one security layer does not expose the entire pipeline.

10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Answer:

I would explain it using a simple analogy:

Scalability:

If the data grows from 1 TB to 10 TB, we can add more machines to the cluster instead of replacing one machine with a much larger one.

Architecture in simple terms:

Large Data → Distributed Storage → Parallel Processing → Combined Results → Business Insights

The key advantages are scalability, fault tolerance, and parallel processing. If one machine fails, replicated data and distributed processing allow the system to continue operating with limited disruption.